directly. How would you like a language that's specifically targeted to text adventures? Given such a language, you could write code like this:

```
me: Actor
    location = westOfHouse
;

westOfHouse : Room 'West of house'
    "You are standing in an open field west of
     a white house, with a boarded front door."
;

+ mailbox : OpenableContainer 'mailbox' 'small mailbox';

++ leaflet : Thing 'leaflet' 'leaflet';
```

This is not a mocked-up example—it's real code. It's written in a language called TADS, specifically designed for creating "interactive fiction" (today's fancier name for text adventures). TADS is an example of a *domain-specific language* (DSL), a language that focuses on a specific problem domain.

The opposite of a DSL is a *general-purpose language* (GPL), such as C++ or Ruby. You can use a GPL to tackle a wide variety of problems, even if it might be more suited to some problems than others. Whenever you write a program, it's up to you to choose between a flexible GPL and a focused DSL.

Let's assume that you decide to go down the DSL route. How would you proceed then?

Using DSLs

If you want a DSL for your own specific problem, you might get lucky. There are hundreds of DSLs around, focusing on a wide range of domains. The UNIX shell is a DSL for gluing command-line utilities together. Microsoft's VBA was designed to extend Excel and other Microsoft Office applications. The make language is a DSL focused on building C programs, and Ant is an XML-based equivalent of make for Java programs. Some of these languages are limited in scope, while others are flexible enough to cross the line into GPL-dom.

What if you can't find a ready-made DSL that fits the domain you're working in? In that case, you can write your own DSL and then use that DSL to write your program. You could say that this process—writing a DSL and then using it—is another take on metaprogramming. It can be a slippery path, though. You'll probably need to define a grammar for your language with a system such as ANTLR or Yacc, which are themselves DSLs for writing language parsers. As the scope of your problem expands, your humble little language